Optimization for Data Science

Implementation and Experimental Results

Authors
Nidhi Rakesh - 2024BCD0006
Akshay VS - 2024BCD0015
Noel Georgi - 2024BCD0023

Submitted to
Dr. Susheel Kumar Joshi

Institution
IIIT Kottayam

Date
25 October 2026

Course Project Report

Abstract

This report documents Python implementations of optimization methods and worked examples drawn from the supplied course lectures. The project covers material from Lectures 1-9, including Lecture 8A, with core calculations written using Python lists and loops. Matplotlib is used for figures; NumPy is imported for the three-dimensional plotting grid in the Peaks example. A saved run contains 32 figures, six CSV files, and a log. The experiments illustrate how conditioning affects gradient descent, how line search and momentum change convergence, how stochastic and adaptive methods behave on regression tasks, and how Newton, BFGS, and KKT methods perform on selected problems. The report maps implemented topics to their code and outputs, records corrections to slide calculations, and distinguishes numerical search results from conclusions supported by convexity or optimality conditions.

1 Introduction

The lecture decks introduce unconstrained and constrained optimization, numerical algorithms, and optimization methods used in simple machine-learning models. This project turns a broad selection of those topics into runnable Python examples. The goal is to compare methods on controlled problems and to check selected worked examples against their stated formulas and values.

The implementation covers foundational derivatives and curvature, gradient-based methods, regression losses, adaptive optimizers, subgradients, Newton's method, BFGS, and KKT systems. The lecture-by-lecture map in Section 3 records which topics were implemented and where the output was saved. The supplied decks are cited in References [1]–[10].

The course sequence can be summarized as follows.

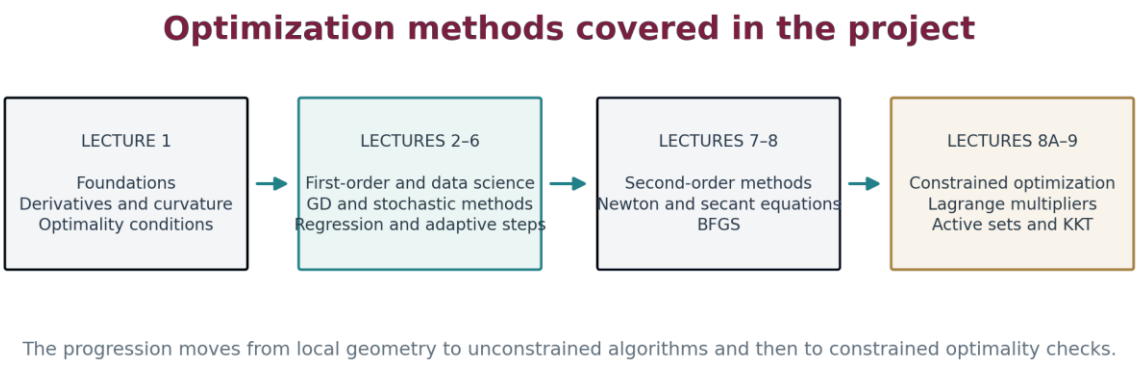


Figure 1. Topic map of the implemented course sequence.

2 Implementation and reproducibility

2.1 Implementation choices

The lecture modules implement vector and matrix operations, gradient updates, line searches, and solver steps with basic Python operations. Shared helpers are in `code/common.py`; the experiment seed and the Lecture 2 and Lecture 3 settings are in `code/config.py`. The runner `code/run_all.py` calls the ten lecture modules and writes output to a timestamped directory under `results/`.

The project uses Matplotlib for plots. `code/lecture1_intro.py` also imports NumPy to construct a mesh for a three-dimensional surface plot. NumPy is therefore an explicit runtime dependency alongside Matplotlib. The repository does not include a dependency manifest.

The experiment seed is 42. The saved run was written to `results/20261025_143100/`; its `log.txt` records the numerical output and seed. All numerical values and figures in this report refer to that saved run.

2.2 Validation approach

The modules print numerical values, residuals, convergence status, and selected comparisons with lecture examples. Some checks are numerical approximations: for example, the subgradient inequalities are sampled on a finite grid, and the Peaks search uses finite grids and multistart gradient descent. The repository has no separate automated test suite, and most logged comparisons do not stop the runner if a value differs from an expected value. A successful full run therefore shows that the examples executed; it is not, by itself, a formal correctness proof.

2.3 Run summary

Item	Recorded value
Runner	`python code/run_all.py`
Seed	42, with additional deterministic seeds noted in module logs
Saved run	`results/20261025_143100/`
Output inventory	32 PNG figures, 6 CSV files, and `log.txt`
Runtime packages	Python 3, Matplotlib, and NumPy

3 Coverage of the lecture decks

This table links the implemented methods to the corresponding sections of the supplied lecture decks.

3.1 Code-to-output traceability

Each lecture module below runs from `code/run_all.py`. The artifact names are relative to the saved run directory `results/20261025_143100/`; numerical summaries and execution messages are appended to `log.txt`.

Source module	What it implements and the files it writes
`code/lecture1_intro.py`	Vector and similarity measures, finite-difference derivatives, Hessian classification, Taylor approximations, optimality checks, knapsack enumeration, and Peaks search. Outputs: `l1_l2_unit_balls.png`, `l1_directional_derivative_contour.png`, `l1_directional_ascent_sweep.png`, `l1_hessian_classification.csv`, `l1_quadratic_forms_unit_circle.png`, `l1_cubic_saddle_contour.png`, `l1_taylor_sin.png`, `l1_taylor_error_vs_alpha.png`, `l1_peaks_minimizers.csv`, `l1_peaks_surface.png`, `l1_peaks_contour.png`.
`code/lecture2_gradient_descent.py`	Fixed-step GD, bounded exact search, Armijo backtracking, momentum, Nesterov, conditioning, and Rosenbrock comparisons. Outputs: `l2_conditioning_convergence.png`, `l2_conditioning.csv`, `l2_line_search_comparison.png`, `l2_rosenbrock_paths.png`, `l2_rosenbrock_convergence.png`, `l2_momentum_comparison.png`.
`code/lecture3_linear_regression.py`	Worked and synthetic linear regression using batch GD, SGD, mini-batches, and the normal equation. Outputs: `l3_linreg_loss_comparison.png`, `l3_gd_loss.csv`.
`code/lecture4_logistic_adaptive.py`	Logistic loss and gradient with GD, Adagrad, RMSProp, and Adam comparisons. Output: `l4_logistic_optimizers.png`.
`code/lecture5_ewma_adam.py`	EWMA weights and smoothing, RMSProp, Adam with bias correction, and objective/regression comparisons. Outputs: `l5_ewma_smoothing.png`, `l5_adam_vs_gd.png`, `l5_rmsprop_vs_adam.png`.
`code/lecture6_subgradient.py`	Subgradient checks and descent, synthetic lasso regularization, and hinge-loss SVM. Outputs: `l6_subgradient_abs_sum.png`, `l6_lasso_convergence.png`, `l6_svm_hinge_boundary.png`.
`code/lecture7_newton.py`	Newton and damped Newton examples, one-dimensional root finding and optimization, conditioning, and Rosenbrock. Outputs: `l7_newton_1d.png`, `l7_newton_rosenbrock.png`.
`code/lecture8_bfgs.py`	Secant pairs, inverse BFGS updates, curvature checks, and quadratic/Rosenbrock comparisons with GD. Outputs: `l8_bfgs_vs_gd_rosenbrock.png`, `l8_bfgs_quadratic.png`.
`code/lecture8a_constrained.py`	Equality-constrained KKT Newton solves, multistart searches, constraint qualification and LICQ examples. Outputs: `l8a_xy_ellipse.png`, `l8a_sum_on_circle.png`, `l8a_two_equality.png`, `l8a_diy_parabola_line.png`, `l8a_kkt_newton_convergence.png`, `l8a_candidates.csv`.
`code/lecture9_kkt.py`	Active-set enumeration, feasibility and multiplier checks, complementarity, and second-order checks for three examples. Outputs: `l9_example1_feasible_region.png`, `l9_kkt_cases.csv`.

The supporting files have distinct roles: `code/run_all.py` invokes all ten lecture modules; `code/common.py` provides shared numerical, logging, plotting, CSV, and run-directory helpers; and `code/config.py` defines the shared seed and selected experiment settings. Each module appends its numerical log to the run's `log.txt`.

3.2 Lecture coverage

Lecture and source sections	Implemented work and output	Coverage notes
1 - Optimization foundations ([1], pp. 7-38)	Norms, dot products, cosine similarity, directional derivatives, finite-difference gradients and Hessians, Hessian eigenvalues and definiteness, Taylor approximations, first- and second-order conditions, a knapsack enumeration, and Peaks-function search. Figures include unit balls, curvature, Taylor error, and Peaks contours/surface.	The bounded form of the Peaks function is used; selected slide calculation corrections are summarized in Section 5.
2 - Gradient descent and variants ([2], pp. 2-22)	Fixed-step GD, bounded exact line search, Armijo backtracking, momentum, and Nesterov. Experiments compare conditioning, line searches, and Rosenbrock paths.	The golden-section line search minimizes over [0, 1]; the result is exact only within that interval. Nesterov diverges with the selected parameters.
3 - Linear regression ([3], pp. 3-18)	Batch GD, SGD, mini-batch SGD, the normal equation, and worked calculations. Outputs include a loss curve and CSV history.	Implemented on the deck examples and seeded synthetic data.
4 - Logistic regression and adaptive steps ([4], pp. 2-10)	Logistic loss and gradient, worked GD/SGD updates, Adagrad, RMSProp, and Adam comparisons on synthetic sparse data.	The optimizer comparison uses a seeded synthetic sparse dataset.
5 - EWMA, RMSProp, and Adam ([5], pp. 2-20)	EWMA smoothing and weights, RMSProp, Adam with bias correction, and function and regression examples.	The reported comparisons depend on the chosen step sizes; Adam does not outperform GD in every example.
6 - Subgradients and regularization ([6], pp. 2-20)	Subgradient checks for absolute value, hinge, and ReLU; a subgradient method; synthetic lasso; and a hinge-loss linear SVM.	The subgradient inequality checks sample a finite grid.
7 - Newton's method ([7], pp. 4-32)	Newton and damped Newton examples, one-dimensional root finding and optimization, conditioning, and a Rosenbrock comparison.	Includes both optimization and root-finding examples.
8 - Secant equations and BFGS ([8], pp. 1-37)	Secant pairs, inverse BFGS update, curvature checks, worked matrix updates, BFGS on a quadratic and Rosenbrock, and comparison with GD.	Curvature checks are applied to the secant pairs before updating the inverse-Hessian estimate.
8A - Constrained optimization ([9], pp. 3-32)	Equality-constrained KKT Newton solves, multistart searches, Lagrange multipliers, constraint qualification and LICQ examples, and an inequality KKT sanity check.	The inequality KKT check complements the equality-constrained worked examples.
9 - KKT conditions ([10], pp. 2-12)	Active-set enumeration for three affine-constrained quadratic examples, feasibility and multiplier checks, complementarity, and second-order sufficiency.	The three examples include feasibility, multiplier, complementarity, and second-order checks.

4 Experimental results

4.1 Gradient descent, conditioning, and line search

For `f(x) = 0.5(a x1^2 + x2^2)`, the step size was reduced as the condition number increased. With the recorded starting point and settings, the iteration count rose from 23 at `a = 1` to 778 at `a = 100`.

Experiment	Result from saved run
Conditioning, `a = 1, 10, 100`	23, 79, and 778 GD iterations
Quadratic line-search comparison	Fixed step: 104 iterations; bounded exact search: 19; Armijo: 39
Rosenbrock GD with fixed step	10,000-iteration cap; final objective `5.853e-05`
Rosenbrock GD with Armijo	10,000-iteration cap; final objective `4.228e-10`
`a = 100` quadratic with momentum	375 iterations, versus 778 for GD
`a = 100` quadratic with Nesterov	Diverged at iteration 155 with the selected step size and momentum parameter

The conditioning plot shows the slower progress at `a = 100`.

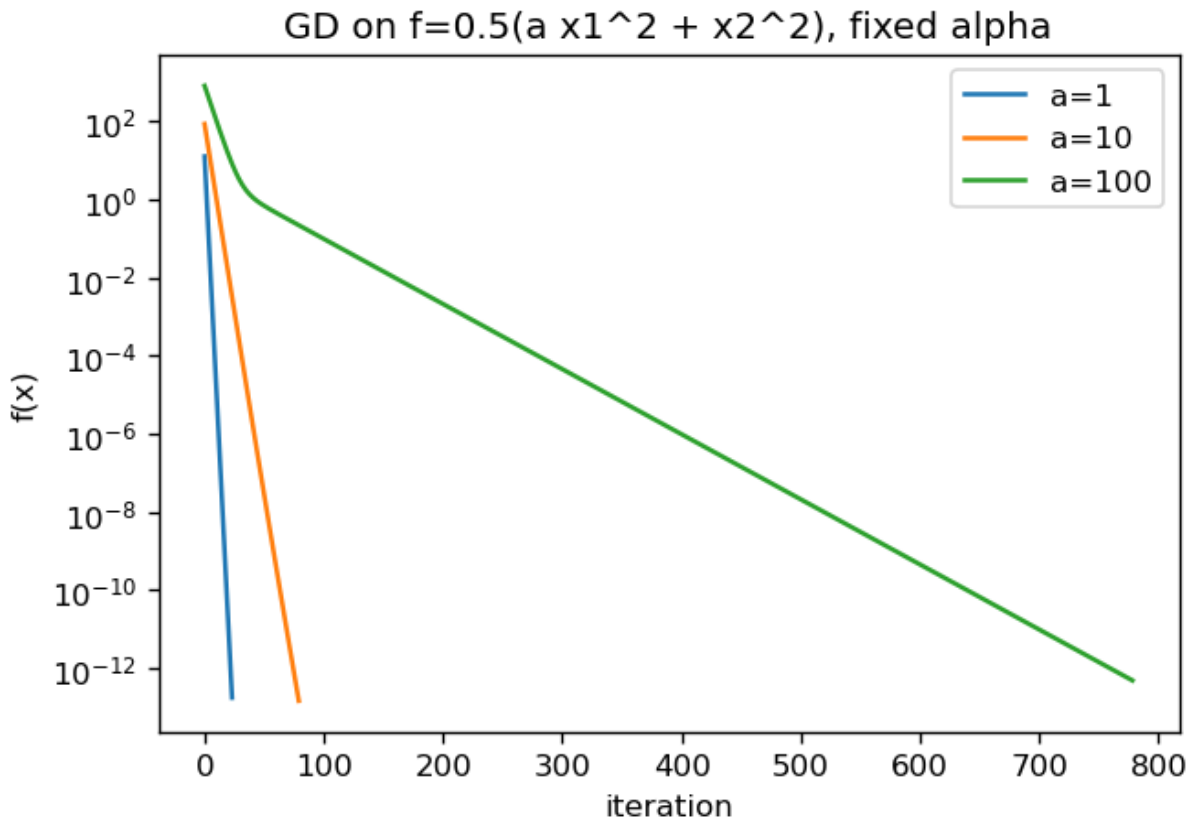


Figure 2. Fixed-step GD needs substantially more iterations as the quadratic becomes more ill-conditioned.

The Nesterov result is parameter-specific. The slide does not specify the momentum parameter used in this experiment, so the divergence should not be interpreted as a general result about Nesterov's method.

4.2 Linear and logistic regression

For the Lecture 3 two-row worked example, the first GD gradient is `(-2.5, -5.0, -7.5)` when the bias is stored first. After two steps, the weights are `(1.0465, 0.59175, 0.63825)` and the loss is `2.319063`. On the seeded synthetic data (`N = 200`, `d = 3`), GD and the normal equation both reach a training loss of `0.005657`; SGD and mini-batch SGD finish at `0.005705` and `0.005685` respectively.

For the Lecture 4 logistic example, the logged losses are `1.294969` and `1.282781` at the first two GD iterations. On synthetic sparse data after 300 iterations, the final cross-entropy values are:

Method	Final loss
GD, `alpha = 0.5`	0.571721

Method	Final loss
Adagrad, `alpha = 0.5`	0.570896
RMSProp, `alpha = 0.01`	0.570901
Adam, `alpha = 0.01`	0.573895

Adagrad and RMSProp finish slightly below GD for this data and these settings; Adam finishes above them. These results do not support a blanket claim that every adaptive method improves the final loss.

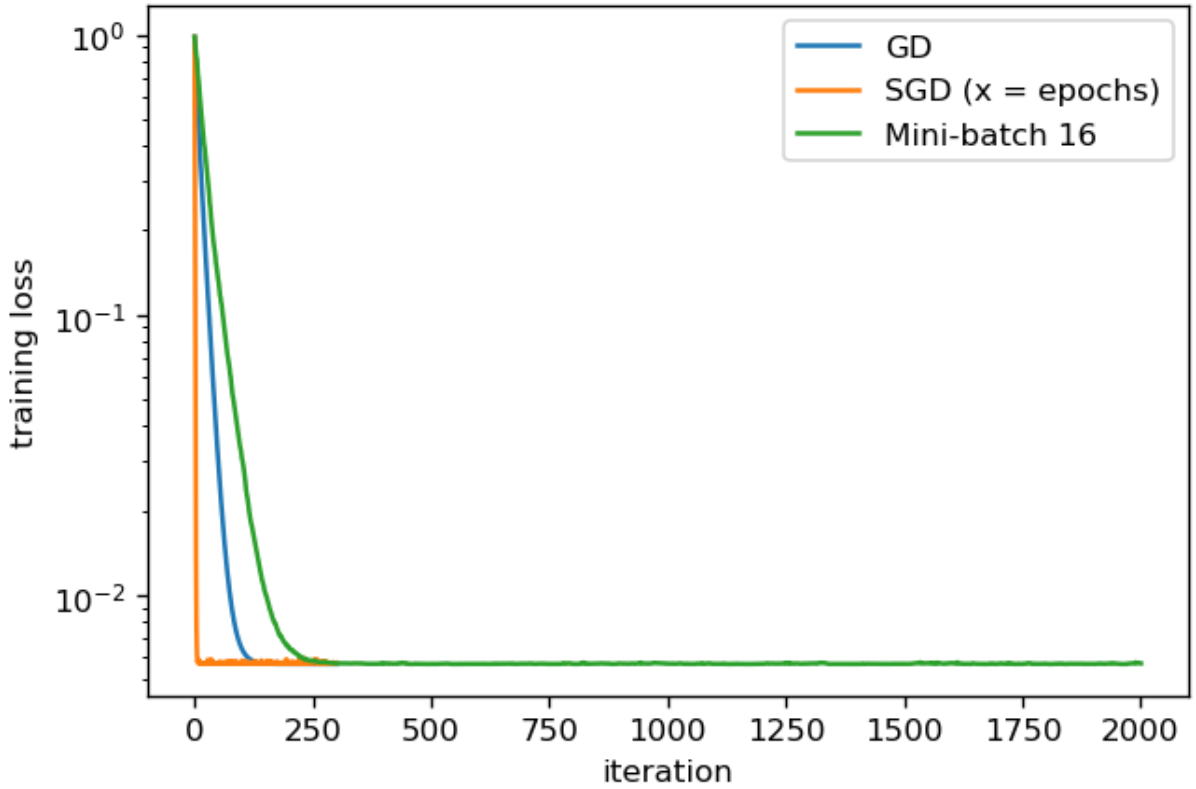


Figure 3. Training loss on the seeded synthetic linear-regression dataset.

4.3 EWMA and adaptive optimizers

The EWMA weights for `beta = 0.1` begin `0.1, 0.09, 0.081, 0.0729`; the first 60 weights sum to `0.998203`. For `f(x) = x1^2 + 2x2^2` from `(1, -1)`, RMSProp and Adam both use `alpha = 0.05` in the 300-step comparison. The final RMSProp objective is `1.875e-03`; Adam reaches `1.197e-13`. In a separate comparison on the same function, fixed-step GD reaches `7.145e-59`, showing that the relative outcome depends on the optimizer and its settings.

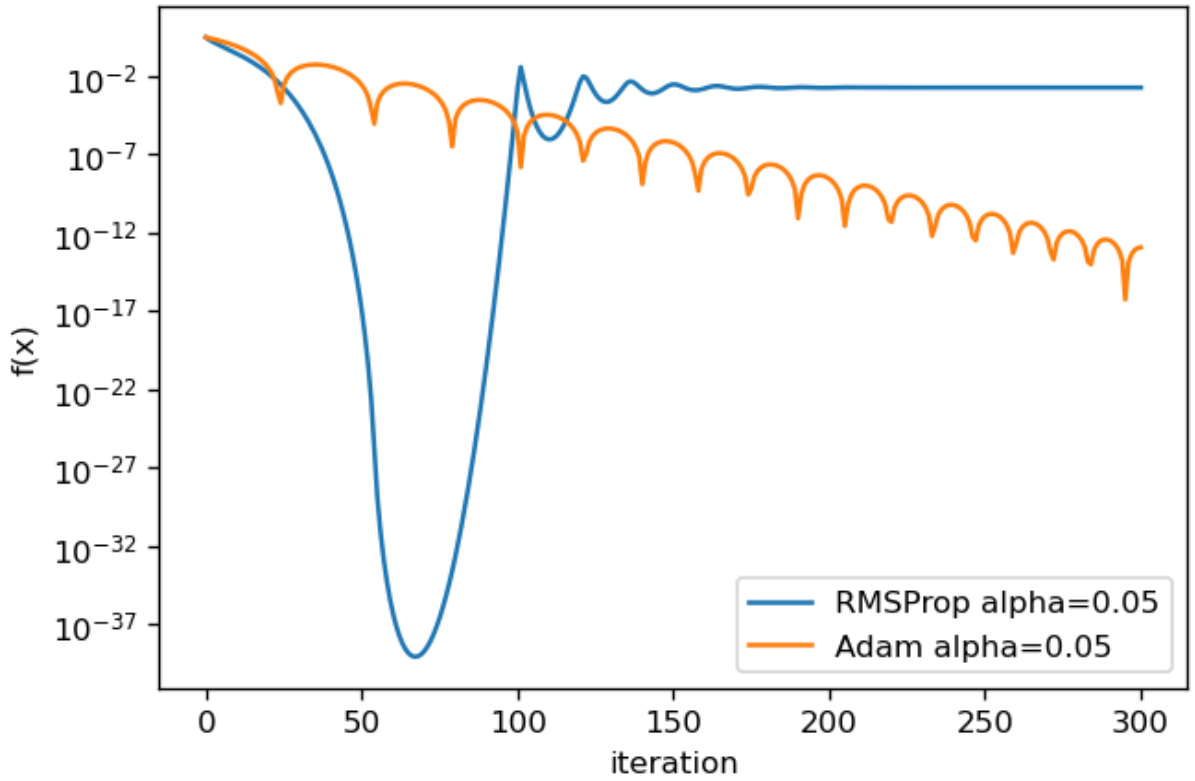


Figure 4. RMSProp briefly approaches a very small objective before its iterates oscillate; Adam decreases more steadily in this run.

4.4 Subgradients, lasso, and hinge-loss SVM

The finite-grid checks classify `g = 0.5` as a valid subgradient of `|x|` at zero and the hinge-loss subgradients at the kink as values in `[-1, 0]`. For `|x1| + 2|x2|`, the best objective recorded after 400 iterations is `0.036510`.

Lasso regularization `lambda`	Best objective	Coefficients below `1e-2`
0.01	0.069188	5
0.1	0.640626	5
0.5	2.880352	5

The synthetic hinge-loss SVM reaches an objective of `0.183146` and training accuracy of `1.000`. These are results for the generated dataset and the selected subgradient step schedule.

4.5 Newton's method and BFGS

On Rosenbrock from `(-1.2, 1)`, undamped Newton converges in six iterations and damped Newton with `lambda = 1` takes 46. Fixed-step GD reaches its 10,000-step cap. BFGS reaches objective `2.958e-18` in 36 iterations, with one skipped curvature update; Armijo GD reaches its cap with final objective `4.228e-10`. BFGS solves the `a = 100` quadratic example in three iterations.

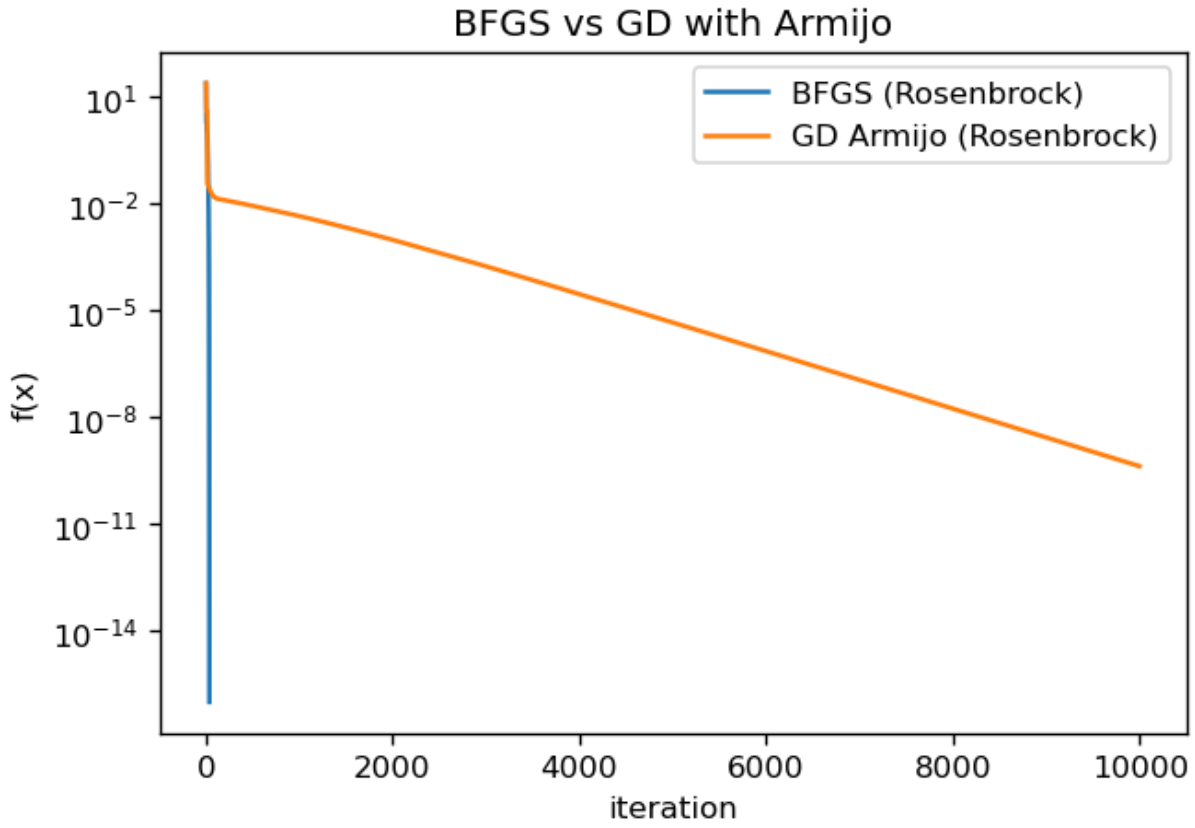


Figure 5. BFGS reaches the Rosenbrock minimizer in far fewer iterations in this experiment.

The inverse BFGS worked update with `H = I`, `delta = (1, 1)`, and `y = (2, 3)` gives `rho = 1/5` and `H' = [[0.92, -0.28], [-0.28, 0.52]]`. The secant equation check returns `H'y = delta` within floating-point precision.

4.6 Constrained optimization

The Lecture 8A multistart solver finds four stationary points for `f = xy` on the ellipse. All 200 starts converge in that case. For the other examples, 199/200 starts converge on the circle problem, 199/200 on the two-equality problem, and 198/200 on the DIY example. The DIY multipliers computed from the stated constraints are `(1, -5)` at both points; this differs from the slide values.

The Lecture 9 active-set solver reports one KKT point for each worked problem:

Example	KKT point	Multipliers	Objective	Second-order result
1	`(5/3, 1/3)`	`lambda1 = 2/3`, `lambda2 = 0`	1	Passes; one-dimensional tangent space with positive reduced Hessian
2	`(3/2, -1/2)`	`lambda = 1`, `mu = -2`	3.5	Critical directions reduce to `{0}`
3	`(0, 4)`	`lambda = 10`, `mu = -4`	13	Critical directions reduce to `{0}`

The three objectives are strictly convex quadratics and the constraints are affine. The KKT candidates therefore identify the global minimizers for these examples. The figure shows the feasible region and the Example 1 optimum.

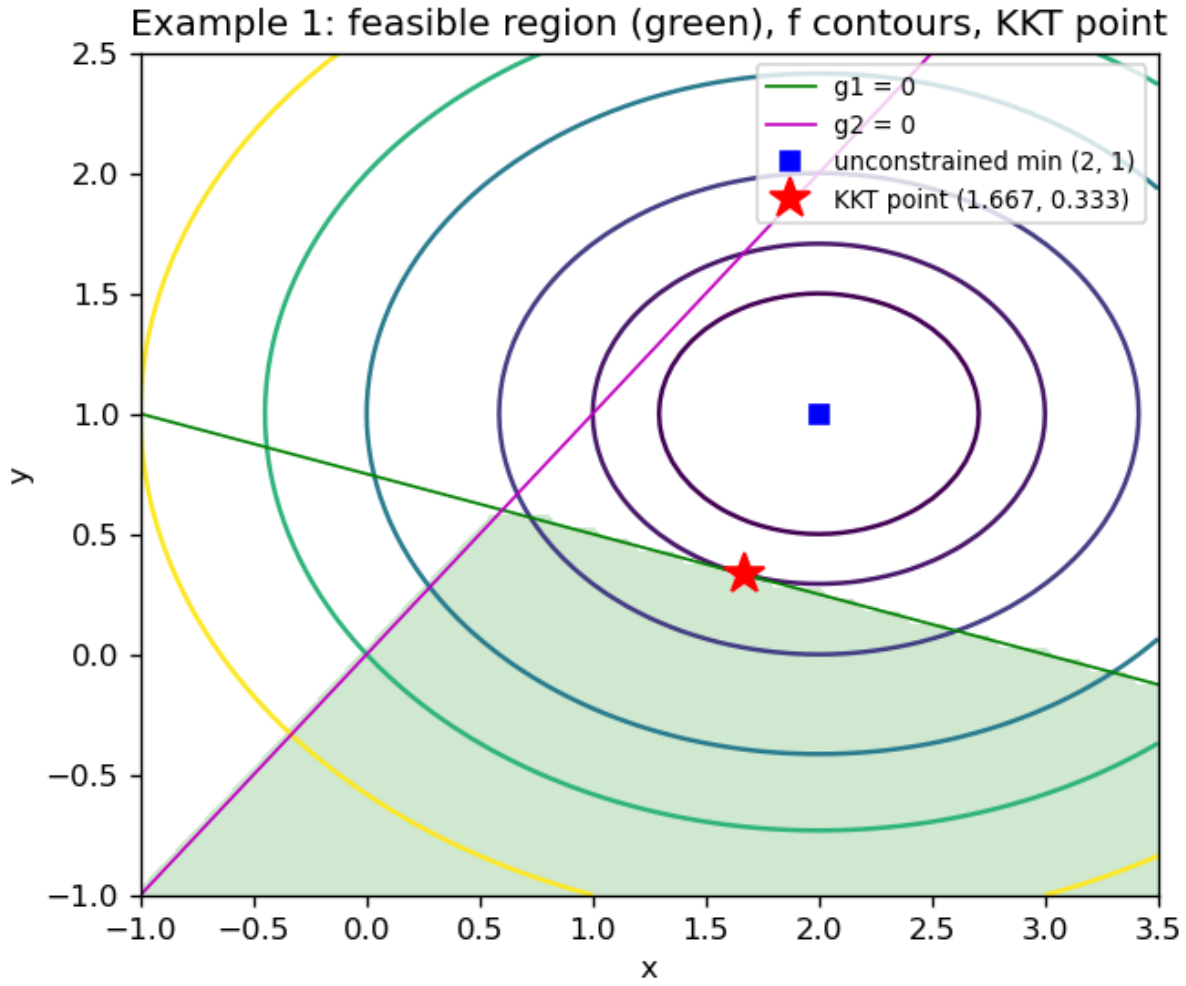


Figure 6. The red star is the KKT point at `(5/3, 1/3)`; the green shading marks the feasible region.

The implemented active-set checks follow this sequence.

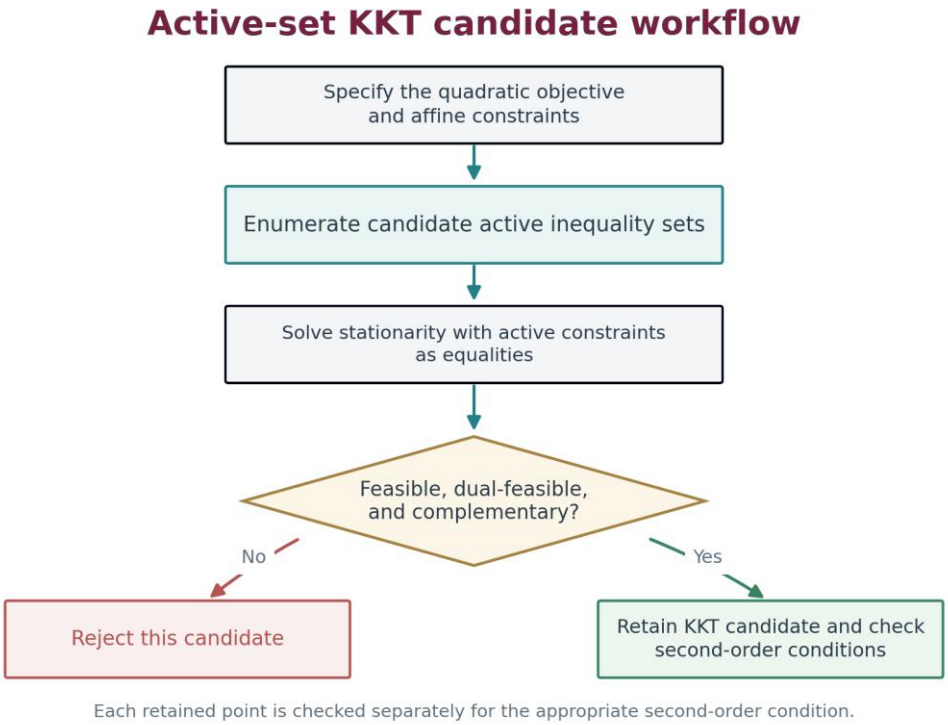


Figure 7. Active-set candidate generation and checks used for the Lecture 9 examples.

5 Corrections to slide calculations

The implementation follows the stated formulas and recomputes values where slide arithmetic or notation is inconsistent.

Lecture	Issue in the slide	Calculation used in the implementation
1	The displayed Hessian, cubic second derivative, eigenvector label, and Peaks exponent contain calculation or notation inconsistencies.	For $f(x_1, x_2) = 6x_1^2$, $H = diag(12, 0)$. The cubic derivative is $\partial^2 f/\partial y^2 = -6x$. Corrected eigenvector labels are used, and the bounded Peaks exponent is $-(x_1^2 + x_2^2)$.
3	The middle component of the worked GD gradient and the subsequent SGD updates do not follow from the stated data.	The gradient component is -5.0; the GD and SGD values are recomputed from the displayed loss.
5	The worked regression loss, gradient scaling, prediction coefficient, and second beta label are inconsistent.	The squared-error loss uses $1/(2N)$, the gradient includes $1/N$, and the second moment parameter is $\beta_2 = 0.99$.
7	The printed gradient and Hessian for Example 1 do not match the stated quadratic.	Newton's step from $(0, 0)$ is $(1.5, 1.5)$, where the stated function equals -13.5.
8A	The DIY example's slide multipliers do not satisfy stationarity for the displayed constraints.	Both stationary points give multipliers $(1, -5)$ under the code's constraint ordering.
9	Several stationarity expressions and Lagrangian constraint labels are inconsistent; Example 2 lists an incorrect x-coordinate.	The equations use gradients of the stated objective and constraints; Example 2 gives $x = 3/2$.

6 Conclusion

The project demonstrates optimization methods across first-order, stochastic, adaptive, second-order, and constrained settings. In the saved run, fixed-step gradient descent slows as conditioning worsens, BFGS reaches the Rosenbrock minimizer in 36 iterations, and outcomes for adaptive and momentum methods depend on their parameters. For the stated strictly convex quadratics with affine constraints, active-set and second-order checks identify the global minimizers. Corrections to slide calculations and code-to-output links make the examples traceable to the saved run.

7 Reproducing the saved run

From the repository root, install Python 3, Matplotlib, and NumPy, then run:

```
python code/run_all.py
```

The runner creates a new timestamped directory under `results/`, writes each figure as a PNG, writes the CSV tables, and appends module logs to `log.txt`. The experiment seed is defined in `code/config.py` and appears in the run log. The run documented here is `results/20261025_143100/`.